Sistemas de Informação



Disciplina: Programação I

Prof. Renê Rodrigues Veloso

Herança forçada

Existem formas de forçar o relacionamento de herança em Java.

Objetivo é obrigar o programador a obedecer certos “contratos” no projeto com a finalidade de manter a organização, aumentar a modularização e fornecer código comum a outras classes.

Classes abstratas

Marcar uma classe como abstrata significa que ela contém métodos abstratos que devem ser definidos em uma classe filha.

Classes abstratas não podem ser instanciadas, ao invés disso devem ser estendidas e os métodos abstratos definidos.

Classes abstratas podem conter uma mistura de métodos concretos e abstratos.

Classes abstratas podem estender de outra classe.

Classes abstratas

```
public abstract class NomeClasse {
   // atributos

   // métodos

   abstract TIPO nomeMetodo();
   . . .
}
```

Classes abstratas

```
public abstract class Animal
{
    protected int idade;
    public abstract void fazerSom();

    public void comer()
    {
        System.out.println("Eu sou um animal e estou comendo.");
    }

    public void mostrarIdade()
    {
        System.out.println(idade);
    }

}
```

Classes abstratas

```
class Cachorro extends Animal
{
    @Override
    public void fazerSom()
    {
        System.out.println("Au Au");
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }


    public static void main(String[] args)
    {
        Cachorro pluto = new Cachorro();
        pluto.setIdade(10);
        pluto.fazerSom();
        pluto.comer();
        pluto.mostrarIdade();
    }
}
```

Ponto final

Da mesma forma, podemos impedir o mecanismo de herança.

Uso da palavra ***final*** modificando a classe:

é o oposto de uma classe abstrata.

Uso da palavra final modificando um método:

impede o override. Implementação final do método.

Classes e métodos final

```
public final class Tigre extends Animal
{
    @Override
    public void fazerSom();
    {
        System.out.println("Roar");
    }
}

public abstract class Mamifero()
{
    public final boolean EHerbivoro()
    {
        return true;
    }
}
```

Interfaces

“Existem diversas situações em Engenharia de Software onde é importante para grupos separados (equipes) de programadores aceitarem um “contrato” que diga como as partes do software interagem. Cada grupo deve ser capaz de escrever seus códigos sem qualquer conhecimento de como os códigos do outro grupo foram escritos. Geralmente, as ***interfaces*** são esses contratos.”

Interfaces

Uma ***Interface*** em Java é definido como um tipo abstrato usado para especificar o comportamento de uma classe. É o “projeto” de uma classe.

Uma ***interface*** contém constantes ***static*** e apenas as assinaturas dos métodos (métodos abstratos).

É usada também para se conseguir vantagens da herança múltipla em Java. Representa um relacionamento do tipo “é um”.

Interfaces em Java

```
modificador interface Nome extends Interface1,...,InterfaceN {

   tipo nomeAtributo1;
   tipo nomeAtributo2;
   . . .


   tipo nomeMetodo1();

   void nomeMetodo2(tipo k, tipo2 t);
   . . .
}
```

Interfaces em Java

```
modificador interface Nome extends Interface1,...,InterfaceN {

   tipo nomeAtributo1;
   tipo nomeAtributo2;
   . . .


   tipo nomeMetodo1();

   void nomeMetodo2(tipo k, tipo2 t);
   . . .
}
```

são public static final por padão

são public por padão

Interfaces em Java

```
modificador interface Nome extends Interface1,...,InterfaceN {

   tipo nomeAtributo1;
   tipo nomeAtributo2;
   . . .


   tipo nomeMetodo1();

   void nomeMetodo2(tipo k, tipo2 t);
   . . .
}
```

são public static final por padão

são public por padão

Interfaces em Java

```
class MinhaClasse extends SuperClasse implements Interface1,...,InterfaceN
{

  ...
}
```

Uma classe pode herdar de apenas uma classe mas pode implementar muitas interfaces!

Interfaces em Java

```
class ClasseExemplo extends ExemploClassePai
implements InterfaceUm, InterfaceDois
{

    @Override
    public void InterfaceUmMetodo() {
    }

    @Override
    public void InterfaceDoisMetodo() {
    }

}
```

Uma classe pode herdar de apenas uma classe mas pode implementar muitas interfaces!

Interfaces em Java

```java
public interface Comestivel {
    public void comer();
}
```

```java
public interface Digestivel {
    public void digerir();
}
```

```java
public class Fruta
implements Comestivel, Digestivel {

    @Override
    public void comer() {
        // ...
    }

    @Override
    public void digerir() {
        // ...
    }
}
```

Métodos Default e Static

Default:

Métodos default podem ser herdados e sobrescritos pelas subclasses.

Static:

Métodos static podem ser herdados, mas não podem ser sobrescritos pelas subclasses.

Exemplos em:
https://www.geeksforgeeks.org/default-methods-java/

Evoluindo interfaces em Java

```
public interface DoIt {
   void doSomething(int i, double x);
   int doSomethingElse(String s);
}
```

Considere essa interface DoIt, já amplamente implementada em todo o seu sistema. Contudo, surgiu a necessidade de incluir mais método abstrato.

Evoluindo interfaces em Java

```
public interface DoIt {
   void doSomething(int i, double x);
   int doSomethingElse(String s);

   boolean didItWork(int i, double x, String s);
}
```

Isso causa um problema… todos as classes que implementam essa interface vão falhar, pois não possuem esse novo método.

Evoluindo interfaces em Java

```
public interface DoIt {
   void doSomething(int i, double x);
   int doSomethingElse(String s);

   boolean didItWork(int i, double x, String s);
}

public interface DoItPlus extends DoIt {

   boolean didItWork(int i, double x, String s);

}
```

Uma solução! As classes que usam DoIt continuarão funcionando.

Evoluindo interfaces em Java

```
public interface DoIt {
   void doSomething(int i, double x);
   int doSomethingElse(String s);

   default boolean didItWork(int i, double x, String s){
        //corpo do método
   }
}
```

Outra solução! Métodos *default* não alteram o contrato e não necessitam modificações e recompilações, podendo ser sobrepostos.

Evoluindo interfaces em Java

```
public interface DoIt {
   void doSomething(int i, double x);
   int doSomethingElse(String s);

   static boolean didItWork(int i, double x, String s){
        //corpo do método
   }
}
```

Outra alternativa… mas fornecer o corpo do método é obrigatório agora.

Exemplo

```
public interface Lista{
  void insere(Integer valor, int pos);
  Integer get(int pos);
  int tamanho();
}
```

Exemplo

```
public class Fila implements Lista{
  private No inicio;
  private No fim;
  private int tamanho;

  public Fila(){
    tamanho = 0;
    inicio = fim = null;
  } . . .
```

Exemplo

```
class Main {
  public static void main(String[] args) {
    Lista l = new Fila();
    l.insere(1);
    l.insere(2);
    l.insere(3);
    int v = l.get();
    System.out.println("Tamanho: "+l.tamanho());
    . . .
```

Exercício

1. Precisamos de um método para imprimir o conteúdo de uma lista, denominado printList, na interface List.

- Qual é a melhor escolha: método default ou método static ?
- Quais alterações devem ser feitas na interface List para que os elementos possam ser impressos, independentemente da implementação?